TripMate AI - Local/Free Stack vs AWS Services

A component-by-component comparison, and where each AWS service would replace the current local one

Project: TripMate AI - multi-agent travel planner (7 agents, MCP tools, Responsible-AI guardrails) **Purpose of this doc:** show, for each part of the app, what it uses **today (free/local)** versus the **AWS service** that could replace it, where it lives in the code, and what you gain or pay.

---

1. At a glance

Dimension	Current (Local / Free)	On AWS
Cost	Rs 0 - runs on your laptop	Pay-per-use (tokens, storage, compute, hourly services)
Setup	`pip install` + Ollama	AWS account, IAM roles, VPC, service config
Scale	One machine, one user at a time	Elastic, many concurrent users
Security / governance	Local files, your responsibility	IAM, VPC, encryption, audit logs, compliance
Best for	Development, demos, learning, privacy	Production, teams, enterprise, scale

Bottom line: the current stack is ideal for building and demoing at zero cost. AWS is the path when the app must run in production for many users with enterprise security, observability and quality controls.

---

2. Component-by-component comparison

2.1 The LLM (the agents' brain)

	Detail
Now (local/free)	Ollama running **Llama 3.2** locally (or the free **Groq** tier).
Where used	travel_agent/llm.py - `get_llm()` powers all 7 agents.
AWS replacement	**Amazon Bedrock** - managed Claude, Llama 3, Amazon Nova/Titan or Mistral via `langchain-aws` `ChatBedrockConverse`.
Why switch	Stronger models (e.g. Claude), no local GPU, high concurrency, data not used for training, region/compliance control.
Cost	Per input/output token (varies by model). Local = free.
Migration effort	**Low** - add a `bedrock` branch in `get_llm()`; no agent code changes.

2.2 Agent orchestration (who runs when)

	Detail
Now	**Google ADK** graph: Guardrail > Supervisor > HITL > specialists > Final.
Where used	travel_agent/graph.py.
AWS replacement	Keep **Google ADK**, hosted on **Bedrock AgentCore Runtime** (serverless agent hosting). Alternatives: Bedrock Agents (multi-agent collaboration) or AWS Strands Agents SDK.
Why switch	Serverless scaling, managed sessions, no server to run; AgentCore is framework-agnostic so your graph is reused.
Cost	Per request / compute-time. Local = free.
Migration effort	**Low-Medium** - package the graph for AgentCore Runtime; logic unchanged.

2.3 Memory (preferences + conversation history)

	Detail
Now	**PostgreSQL** (or in-memory) via `langgraph-checkpoint-postgres`.
Where used	travel_agent/memory.py.
AWS replacement	**Bedrock AgentCore Memory** (managed short- & long-term memory), or **DynamoDB / Aurora** behind the checkpointer.
Why switch	Fully managed, durable, scalable; no DB to operate; built-in long-term user memory.
Cost	Per stored/retrieved item or DB capacity. Local in-memory = free.
Migration effort	**Medium** - swap the checkpointer/store backend.

2.4 Tools & MCP

	Detail
Now	Local **MCP servers** (weather, flights, search) called in-process.
Where used	mcp_servers/, travel_agent/mcp_client.py.
AWS replacement	**Bedrock AgentCore Gateway** - turns APIs/Lambdas into governed **MCP** tools with auth, throttling and logging.
Why switch	Centralised, secured, observable tool access; same MCP standard you already use.
Cost	Per tool invocation / Lambda + Gateway. Local = free.
Migration effort	**Low-Medium** - host the same tools behind Gateway.

2.5 Responsible AI / Guardrails

	Detail
Now	**Llama Guard** (optional) + **regex rules**: safety scan, PII redaction, injection detection, disclaimers.
Where used	travel_agent/responsible_ai.py + agents/guardrail.py.

	Detail
AWS replacement	**Amazon Bedrock Guardrails** - managed content filters, **denied topics** (enforce worldwide), **built-in PII detection & masking**, word filters, and **contextual grounding** checks.
Why switch	Centrally configured, versioned, audited, consistently enforced across every agent; less custom code to maintain.
Cost	Per "text unit" evaluated. Local rules = free; Llama Guard local = free.
Migration effort	**Medium** - add a `bedrock_guardrail` engine in the existing RAI layer / attach a guardrail ID to the model.

2.6 Knowledge grounding - RAG / GraphRAG (new capability)

	Detail
Now	**None** - agents rely on the LLM + live web/API data; no curated knowledge base.
Where it would help	Hotel, Itinerary and Final agents (agents/).
AWS addition	**Bedrock Knowledge Bases** (RAG over curated India travel content in **S3** + **OpenSearch Serverless** vectors). **GraphRAG** via **Amazon Neptune** for relationships: *destination* > *attractions* > *nearby* > *transport*.
Why add	Grounds answers in trusted content, cuts hallucination, enables "why" relationship queries - strengthens the grounding/accuracy pillar.
Cost	Vector store (OpenSearch Serverless has a minimum monthly floor), Neptune hourly, embedding tokens.
Migration effort	**Medium-High** - curate content, build the knowledge base, wire retrieval into agents.

2.7 Authentication (login/register)

	Detail
Now	**SQLite** user store + **PBKDF2** password hashing.
Where used	travel_agent/auth.py, login_ui.py.
AWS replacement	**Amazon Cognito** (user pools, MFA, social login) / **AgentCore Identity**.
Why switch	Managed auth, MFA, token-based security, scales to many users.
Cost	Per monthly active user (free tier for small volumes).
Migration effort	**Medium**.

2.8 External data tools (weather, flights, search, places)

	Detail
Now	**Open-Meteo** (free weather), **AviationStack**, **Tavily**, **Google Places**.
Where used	travel_agent/tools/.

	Detail
On AWS	Usually **keep these external APIs** (they're specialist data sources), wrapped as **Lambda** tools behind AgentCore Gateway for governance.
Why	AWS doesn't replace a flight/weather data provider; it governs and secures the calls.
Migration effort	**Low** (optional).

2.9 Observability & evaluation

	Detail
Now	A simple `llm_calls` counter and the Responsible-AI log (`rai_log`); **no formal evaluation**.
AWS addition	**AgentCore Observability** + **CloudWatch / X-Ray** for tracing; **Bedrock Evaluations** (LLM-as-judge, RAG groundedness) plus open-source **RAGAS / LangSmith** for quality & safety scoring.
Why add	Trace multi-agent runs, catch regressions, and *prove* itinerary quality, guardrail catch-rate and PII-redaction - the evidence a production review needs.
Cost	Per evaluation job / log volume.
Migration effort	**Medium**.

2.10 User interface

	Detail
Now	**Streamlit** app on localhost.
Where used	streamlit_app.py.
AWS hosting	Same Streamlit app on **AWS App Runner** or **ECS Fargate** (or a React front-end on **Amplify**).
Migration effort	**Low**.

---

3. Side-by-side architecture

```
CURRENT (free / local)                        AWS (production)
----------------------                        ----------------
Streamlit on laptop                           Streamlit on App Runner / ECS
Google ADK (in process)                        Google ADK on AgentCore Runtime
Ollama Llama 3.2 / Groq          -- LLM -->    Amazon Bedrock (Claude / Llama / Nova)
Llama Guard + regex rules     -- Guardrails -> Amazon Bedrock Guardrails
Local MCP servers              -- Tools ---->  AgentCore Gateway (MCP) + Lambda
PostgreSQL / in-memory        -- Memory ---->  AgentCore Memory / DynamoDB
SQLite + PBKDF2 login         -- Auth ----->   Amazon Cognito / AgentCore Identity
(none)                        -- Knowledge ->  Bedrock Knowledge Bases + Neptune (GraphRAG)
llm_calls counter             -- Observe --->  AgentCore Observability + CloudWatch
(none)                        -- Evaluate ->   Bedrock Evaluations + RAGAS / LangSmith
External APIs (Open-Meteo, AviationStack, Tavily, Google) - kept in both; governed via Gateway on AWS
```

---

4. Cost & when to use which

Use case	Recommended stack
Development, demos, learning, full privacy, zero budget	**Current local/free stack**
A few internal users, occasional use	Hybrid - Bedrock for the LLM + Guardrails, keep the rest local
Production, many users, enterprise security & audit, quality SLAs	**Full AWS** (Bedrock + AgentCore + Knowledge Bases + Evaluations)

AWS costs are **usage-based**: LLM tokens, guardrail text units, vector store (OpenSearch Serverless has a monthly minimum), Neptune (hourly), memory/storage, and compute. The local stack has **no per-use cost**. Always check current AWS pricing for your region before committing.

---

5. Recommendation

1. **Keep the current free/local stack as the default** for development and demos.
2. **First AWS step (highest value, lowest effort):** move the **LLM to Amazon Bedrock** and the **safety layer to Bedrock Guardrails** - both plug into existing files (llm.py, responsible_ai.py) with minimal change.
3. **Then, for production:** host the Google ADK app on **AgentCore Runtime**, move memory to **AgentCore Memory**, tools to **Gateway**, and auth to **Cognito**.
4. **For accuracy & trust:** add **Bedrock Knowledge Bases / GraphRAG** and **Evaluations**.

The design does not need to be rebuilt - AWS adoption is mostly **swapping backends behind the same architecture** (multi-agent + MCP + guardrails + memory), which is why the migration effort per component is mostly Low-Medium.

TripMate AI - Local vs AWS comparison. The app remains free and local by default; AWS is the optional enterprise/production upgrade path.